Hisa Go Partner API Integration Guide

1. Introduction **3**
- Available Endpoints 3
  - 1. Airtime 3
  - 2. Data 4
  - 3. Transactions 5
  - 4. Electricity 5
  - 5. Cable TV 6
  - 6. Wallet and Balance 8

2. Authentication **8**

3. API Endpoints **8**
- Api key 8
- Airtime 9
  - Request Body Parameters 9
    - Response 10
  - Transaction Status Codes 10
  - Notes 11
- Data 12
  - Response Field Descriptions 13
  - Notes 14
- Data plans 15
  - Usage Options 15
  - Response Parameters (Overview) 16
- Airtime/Data Transaction History 20
- Transaction details 22
  - Usage 22
  - Notes 23
- Transaction details by ref 24
  - Response Parameters (Overview) 24
- Vend Glo gift data 27
  - Use Cases 27
  - Request Body Parameters 27
  - Response Parameters 27
- Glo gift data plans 30
  - Use Cases 30
- Electricity Billers 31
  - Use Cases 31
  - Response Parameters 32

Notes....33
Electricity Verify....33
Use Cases....33
Request Body Parameters....34
Response Parameters....34
Example Request....36
Example Response (Successful Verification)....36
Example Response (Validation Error)....37
Notes....37
Electricity Vend....37
Use Cases....38
Request Body Parameters....38
Example Request....39
Response Parameters....39
Example Response (Successful Vend)....41
Notes....42
Cable TV Billers....43
Description....43
Use Cases....43
Response Parameters....44
Example Response....44
Notes....45
Cable TV Biller Products....45
Description....45
Use Cases....45
Query Parameters....46
Response Parameters....46
Example Response....46
Notes....47
Cable TV Verification....47
Description....47
Use Cases....48
Request Body Parameters....48
Example Request....48
Response Parameters....48
Example Response (Successful Verification)....49
Example Response (Invalid Smartcard)....50
Notes....50
Cable Vend Service....51
Description....51

Use Cases 51
Request Body Parameters 51
Example Request 52
Response Parameters 53
Example Response (Successful Vend) 54
Example Response (Failed Vend) 55
Notes 55
Balance Check 56
Request 56
Response 56
Field Descriptions 57
Usage Notes 57
4. Example Integration Flow 57
5. Error Codes and Troubleshooting 58
6. Frequently Asked Questions (FAQ) 58
7. Support and Contact 59

1. Introduction

The Hisa Go Partner API enables businesses and developers to integrate airtime, data, electricity, and cable vending services seamlessly into their platforms.

It supports real-time transactions and wallet management, helping partners deliver fast and reliable digital value services to their end users.

This documentation outlines how to authenticate, interact with available endpoints, and follow the recommended integration flow to ensure smooth deployment and minimal errors.

- Base URL: https://api.airtimereport.com/api/

Available Endpoints

1. Airtime

- `POST /airtime` — Purchase airtime for any supported network.

```
curl --location 'https://api.airtimereport.com/api/airtime' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer ******' \
--data '{
    "msisdn": "08030000000",
    "amount": 1000,
 "network": "mtn"
}'
```

2. Data

- `GET /data-plan` — Retrieve all available data bundles by network.

```
curl --location
'https://api.airtimereport.com/api/data-plan?network=mtn' \
--header 'Authorization: Bearer ******'
```

- `POST /data` — Purchase a data bundle for a customer.

```
curl --location 'https://api.airtimereport.com/api/data' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer *******' \
--data '{
    "msisdn": "07030000000",
    "bundle_code": "M020010",
    "network": "mtn",
    "ref": "test_local"}'
```

- `GET /glo-gift-plan` — Retrieve available Glo gift data bundles.

```
curl --location 'https://api.airtimereport.com/api/glo-gift-plan' \
--header 'Authorization: Bearer *******'
```

- `POST /glo-gift-data` — Vend Glo gift data.

```
curl --location 'https://api.airtimereport.com/api/glo-gift-data' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer *******' \
--data '{
    "msisdn": "07030000000",
    "bundle_code": "A01",
    "volume": 3,
    "ref": "vend_glo_1234"
}'
```

3. Transactions

- `GET /transaction/:transactionID` — Retrieve transaction details by transaction ID.

```
curl --location
'https://api.airtimereport.com/api/transaction/34517237351671439936' \
--header 'Authorization: Bearer *******'
```

- `GET /transaction-by-ref/:ref` — Retrieve transaction details by reference ID.

```
curl --location
'https://api.airtimereport.com/api/transaction-by-ref/test-mtn-may-17
-3'
```

4. Electricity

- `GET /bills/electricity/billers` — Retrieve a list of electricity distribution companies (DISCOs).

```
curl --location
'https://api.airtimereport.com/api/bills/electricity/billers' \
--header 'Authorization: Bearer *******' \
--data ''
```

- `POST /bills/electricity/verify` — Verify a customer's meter number and account type (Prepaid/Postpaid).

```
curl --location
'https://api.airtimereport.com/api/bills/electricity/verify' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer ******' \
--data '{
    "amount":100000,
    "meter": "1234567890",
    "product": "Prepaid"
}'
```

- `POST /bills/electricity/vend` — Vend electricity tokens for verified meters.

```
curl --location
'https://api.airtimereport.com/api/bills/electricity/vend' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer *******' \
--data '{
   "amount":50000,
    "biller": "EKO",
    "meter": "0101150481949",
    "product": "Prepaid",
    "name": "ORITSE EBINEMI",
    "address": "  Alhaji Marufu Ali Owe Street Plt8b Agungi Lekki",

    "ref": "3643011630212321280"
}'
```

5. Cable TV

- `GET /bills/cable/billers` — Retrieve available cable billers.

```
curl --location
'https://api.airtimereport.com/api/bills/cable/billers' \
--header 'Authorization: Bearer ******' \
--data ''
```

- `GET /bills/cable/biller-products?biller=GOTV` — Retrieve available cable subscription products.

```
curl --location
'https://api.airtimereport.com/api/bills/cable/biller-products?biller
=GOTV' \
--header 'Authorization: Bearer *******' \
--data ''
```

- `GET /bills/cable/verify` — Verify customer's smartcard number.

```
curl --location
'https://api.airtimereport.com/api/bills/cable/verify' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer *******' \
--data '{
    "biller": "DSTV",
    "number": "1234567890",
    "product": "2336"
}'
```

- `POST /bills/cable/vend` — Vend a cable subscription for verified customers.

```
curl --location 'https://api.airtimereport.com/api/bills/cable/vend'
\
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer *********' \
--data '{
    "biller": "GOTV",
    "number": "8072156259",
    "product": "ng_gotv_gotvnj1",
    "name": "FATIMA HUSSEIN",
    "address": "8072156259",
    "ref": "123456789000"
}'
```

6. Wallet and Balance

- `GET /balance` — Retrieve current account or wallet balance before initiating transactions.

```
curl --location 'https://api.airtimereport.com/api/me/balance' \
--header 'Authorization: Bearer
creditpal@gmail.com.3CK6KAS3JRUS6WT4NWB4QS2N4WN6X5XP' \
--data ''
```

2. Authentication

Each API request must include a valid authorization token. Use the 'Authorization' header as shown below:

- Authorization: Bearer <your_access_token>
- Content-Type: application/json

3. API Endpoints

Api key

Method: GET

URL: {{host}}/me/api-key

Response Example (Api key):

```
{
   "data": {
      "api_key": "jondoe@mail.com.ABCDEFGHIJKLMNOP"
   },
   "message": "Successful"
}
```

Airtime

The **Airtime Purchase API** allows partners to initiate airtime top-up transactions for a specified mobile number (`msisdn`).
It processes the airtime purchase request and returns the transaction details, including the status and reference information.

Request Body Parameters

Parameter	Type	Required	Description
`msisdn`	string	Yes	The recipient's mobile phone number in international or local format.
`amount`	integer	Yes	Amount of airtime to purchase, expressed in **kobo** (₦1 = 100 kobo).
`ref`	string	No	A unique transaction reference provided by the client for idempotency and tracking.
`network`	string	Yes	The mobile network provider (e.g., `"mtn"`, `"airtel"`, `"glo"`, `"9mobile"`).

Response

Field	Type	Description
`transaction_id`	string	Unique identifier for the transaction on the API platform.
`amount`	integer	Amount of airtime purchased.
`network`	string	Network provider of the recipient.
`msisdn`	string	The recipient's mobile number.
`type`	string	Type of transaction (`"Airtime"`).
`external_ref`	string	The client-provided reference ID.

`status_code`	string	Status of the transaction represented in codes, e.g, (“00”,”06”, etc.)
`status`	string	Status of the transaction (`"Successful"`, `"Failed"`, `"Pending"`, `"TelcoError"`).
`created_at`	string	Timestamp of when the transaction was created.

Transaction Status Codes

Status Code	Status	Meaning
06	Pending	Transaction still pending
00	Successful	Transaction
01	Failed	Transaction failed on the telco side.
02	InvalidMsisdn	Invalid or barred MSISDN
03	InvalidAmount	Amount requested is invalid
04	TelcoError	Error while vending on the telco side.
05	InternalError	Internal processing error.
06	InvalidTelco	Invalid telco specified in request

Notes

- Always ensure that `ref` values are **unique per request** to prevent duplicate airtime top-ups.
- The `status` field in the response can be used to determine the success or failure of the transaction.
- Transactions are typically processed in real-time, but network latency may cause slight delays.

Method: POST

URL: {{host}}/airtime

Request Body Example:

```
{
  "msisdn": "08023683516",
  "amount": 1000, // in kobo
  "ref": "test-mtn-sept-03-25",
  "network": "mtn"
}
```

Response Example (airtime):

```
{
  "data": {
    "transaction_id": "370179020458823680",
    "amount": 50,
    "network": "airtel",
    "msisdn": "07030000000",
    "type": "Airtime",
    "external_ref": "12343",
    "status": "Successful",
    "created_at": "2023-10-19T12:59:16.072+01:00"
  }
}
```

Response Example (unauthorized):

Unauthorized

Response Example (failed):

```
{
  "data": {
    "id": 9,
    "transaction_id": "370172342984249344",
    "amount": 100,
    "network": "Airtel",
    "msisdn": "07030000000",
    "type": "Airtime",
    "external_ref": "1234",
    "status": "TelcoError",
    "created_at": "2023-10-19T12:32:44.037+01:00"
  }
}
```

Response Example (in suficient balance):

```
{
   "message": "Insufficient Balance"
}
```

Data

The **Data Purchase API** enables partners to initiate a mobile data bundle purchase for a specific customer (`msisdn`) on a supported network.

It accepts a valid data bundle code and returns detailed transaction information, including status, bundle description, and timestamp.

This endpoint is ideal for integrating automated or customer-facing data top-up systems.

Request Body Parameters

Parameter	Type	Required	Description
`msisdn`	string	Yes	The recipient's mobile phone number in local or international format.
`bundle_code`	string	Yes	The unique code identifying the data plan or bundle to purchase. You can get the from the /data-plan endpoint . In the /data-plan api bundle.item is the budle_code parameter.
`network`	string	Yes	The target mobile network (e.g., `mtn`, `airtel`, `glo`, `9mobile`).
`ref`	string	Yes	A unique transaction reference string generated by the client for tracking and idempotency.

Response Field Descriptions

Field	Type	Description
`transaction_id`	string	Unique transaction identifier assigned by the platform.
`amount`	integer	Cost of the data bundle purchased.
`network`	string	Network provider where the bundle was activated.
`msisdn`	string	Recipient's mobile number.
`type`	string	Type of transaction (`Data`).
`data_bundle_code`	string	Internal or system code representing the data bundle.
`data_bundle_desc`	string	Readable description of the purchased data plan.
`external_ref`	string	The reference value you provided in the request.
`status`	string	Transaction outcome (`Successful`, `Pending`, or `Failed`).
`created_at`	string	ISO 8601 timestamp of when the transaction was processed.

Notes

- Each transaction `ref` must be unique to prevent duplicate data bundle purchases.
- Ensure that the `bundle_code` corresponds to an active and supported data plan on the specified network from the data-plans endpoint.
- Use the `status` field to verify whether the data top-up succeeded or failed before notifying end users.
- Network-level errors (e.g., `"TelcoError"`) may occur during congestion — retry with caution.

Method: POST

URL: {{host}}/data

Request Body Example:


```
{
  "msisdn": "07030000000",
  "bundle_code": "M020010",
  "network": "mtn",
  "ref": "test_local"
}
```


Response Example (data):


```
{
  "data": {
    "transaction_id": "384282286704562176",
    "amount": 100,
    "network": "mtn",
    "msisdn": "07030000000",
    "type": "Data",
    "data_bundle_code": "A02",
    "data_bundle_desc": "DataPlan 100MB Daily",
    "external_ref": "test_local",
```

```
    "status": "Successful",
    "created_at": "2023-11-27T10:00:36.771Z"
  }
}
```


Data plans

This endpoint retrieves a comprehensive list of available data plans across different mobile networks such as MTN, Airtel, Glo, and 9mobile.
Developers can use it to fetch all plans or filter plans by specific networks.

Each plan is grouped by **bundle categories** (e.g., Daily, Weekly, Monthly) and contain details such as data size, validity period, cost e.t.c .
The endpoint helps display available options to end-users before making a purchase via the `/data` endpoint.

Usage Options

- **All Networks:**
  Returns available data plans for all networks.

- **Specific Network:**
  Append a query parameter such as `?network=mtn` to retrieve plans for one network only.

Response Parameters (Overview)

Field	Type	Description
`name`	string	The category of the data plan (e.g., Daily Data Plan, Monthly Bundle).
`network_id`	string	The network provider's code (e.g., `mtn`, `airtel`).
`bundle`	array	A list of data plans under the category.
`bundle[].item`	string	The unique bundle code for the plan.

`bundle[].description`	string	Description of the data package.
`bundle[].validity`	string	Validity duration (e.g., `1 Day`, `30 Days`).
`bundle[].cost`	number	The plan's cost in local currency.

Method: GET

URL: {{host}}/data-plan

Response Example (All):


```
{
 "data": {
  "airtel": [
   {
    "name": "Daily Data Plan",
    "network_id": "airtel",
    "bundle": [
     {
      "item": "A02",
      "description": "100MB",
      "validity": "1 Day",
      "cost": 99
     },
     {
      "item": "A03",
      "description": "200MB",
      "validity": "3 Days",
```

```
      "cost": 199.03
     }
    ]
  },
  {
   "name": "Monthly Data Plan",
   "network_id": "airtel",
   "bundle": [
    {
     "item": "A09",
     "description": "1.5GB",
     "validity": "30 Days",
     "cost": 999
    }
   ]
  }
],
"mtn": [
 {
  "name": "Daily Data Plan",
  "network_id": "mtn",
  "bundle": [
   {
    "item": "M0901",
    "description": "100MB Daily Plan",
    "validity": "1 Day",
```

```
          "cost": 100
        },
        {
          "item": "M020010",
          "description": "2.5GB 2-Day Plan",
          "validity": "2 Days",
          "cost": 600
        }
      ]
    }
  ]
 }
}
```


Response Example (Network):


```
{
  "data": [
   {
     "name": "Daily Data Plan",
     "network_id": "mtn",
     "bundle": [
      {
        "item": "M0901",
        "description": "100MB Daily Plan",
        "validity": "1 Day",
```

```
        "cost": 100
      },
      {
        "item": "M0902",
        "description": "200MB 3-Day Plan",
        "validity": "1 Day",
        "cost": 200
      }
    ]
  },
  {
    "name": "Monthly Data Plan",
    "network_id": "mtn",
    "bundle": [
      {
        "item": "M0903",
        "description": "1.5GB Monthly Plan + YouTube Bonus",
        "validity": "30 Days",
        "cost": 1200
      }
    ]
  }
 ]
}
```

Airtime/Data Transaction History

Method: GET

URL: {{host}}/history?transaction_id=&msisdn=&ref=&page=

Response Example (history):


```
{
   "data": {
      "pagination": {
         "current_page": 1,
         "next_page": 2,
         "next_page_url":
"/api/history?msisdn=&page=2&per_page=5&ref=&transaction_id=",
         "prev_page": 0,
         "prev_page_url": "",
         "total_pages": 5
      },
      "records": [
         {
            "transaction_id": "3446352461217665024",
            "amount": 5000,
            "network": "mtn",
            "msisdn": "07030000000",
            "type": "Airtime",
            "data_bundle_code": "",
            "data_bundle_desc": "",
            "external_ref": "test-mtn-may16-2",
            "status": "Successful",
            "created_at": "2024-05-16T17:45:19.587+01:00",
            "commission_amount": 160
         },
         {
            "transaction_id": "3446352382285058048",
            "amount": 5000,
            "network": "mtn",
            "msisdn": "07030000001",
            "type": "Airtime",
            "data_bundle_code": "",
            "data_bundle_desc": "",
            "external_ref": "test-mtn-may16-1",
            "status": "Successful",
```

```
    "created_at": "2024-05-16T17:45:00.774+01:00",
    "commission_amount": 160
  },
  {
    "transaction_id": "3445881819728973824",
    "amount": 10000,
    "network": "mtn",
    "msisdn": "07030000002",
    "type": "Airtime",
    "data_bundle_code": "",
    "data_bundle_desc": "",
    "external_ref": "test-mtn4",
    "status": "Successful",
    "created_at": "2024-05-15T10:35:09.919+01:00",
    "commission_amount": 320
  },
  {
    "transaction_id": "3445879708194705408",
    "amount": 10000,
    "network": "mtn",
    "msisdn": "07030000000",
    "type": "Airtime",
    "data_bundle_code": "",
    "data_bundle_desc": "",
    "external_ref": "test-mtn3",
    "status": "Successful",
    "created_at": "2024-05-15T10:26:46.49+01:00",
    "commission_amount": 0
  },
  {
    "transaction_id": "3445878021912530944",
    "amount": 35000,
    "network": "mtn",
    "msisdn": "07030000003",
    "type": "Data",
    "data_bundle_code": "M01805",
    "data_bundle_desc": "1GB Daily Plan + 3mins.",
    "external_ref": "test-data-3",
    "status": "Successful",
    "created_at": "2024-05-15T10:20:04.448+01:00",
    "commission_amount": 0
  }
],
```

```
      "total_records": 23
    }
}
```


Transaction details

This endpoint retrieves the detailed status and metadata of a specific transaction using its unique `transactionID`.
It provides insight into both the system-level transaction record and the network operator's (telco) response log.

This is useful for verifying the outcome of an airtime, data, or other value-added service transaction, especially for reconciliation or troubleshooting purposes.

Usage

Replace the `:transactionID` parameter in the URL with the actual transaction ID obtained from a prior purchase response.

Method: GET

URL: {{host}}/transaction/:transactionID

Response Example (transaction status):


```
{
   "data": {
      "telco_log": {
         "error": "",
         "response_http_code": 200,
         "response_status_code": "200",
         "response_transaction_id": "R231024.1306.230064",
         "status": "Successful",
         "transaction_id": "371992695607922688"
      },
      "transaction": {
         "transaction_id": "371992695607922688",
         "amount": 100,
         "network": "Airtel",
         "msisdn": "07030000000",
```

```
        "type": "Airtime",
        "external_ref": "1234",
        "status": "Successful",
        "created_at": "2023-10-24T13:06:09.924+01:00"
      }
    },
    "status": "Successful"
}
```


Notes

- This endpoint helps verify the **real-time delivery** or **final status** of a transaction.
- The `telco_log` section is particularly helpful for confirming responses directly from the telecom operator.
- If the transaction is still being processed, the status may temporarily return as `"Pending"`.
- Useful for integration with transaction history pages, support tools, or automated reconciliation scripts.

Transaction details by ref

This endpoint retrieves the complete details of a specific transaction using your custom external reference (`ref`) instead of the system-generated transaction ID.
It provides both the system record and the telecom provider's response for the transaction, making it useful for reconciliation, verification, and post-transaction audits.

Use this when you want to track transactions based on your own reference format instead of the platform's internal transaction ID.

Response Parameters (Overview)

Field	Type	Description
`data.reversal_log`	object / null	Contains details of any reversal action if applicable; `null` if none.

`data.telco_log`	object	Contains response data from the telecom operator.
`data.telco_log.response_transaction_id`	string	Unique transaction identifier returned by the telco.
`data.telco_log.status`	string	Status as confirmed by the telco (e.g., `Successful`, `Failed`).
`data.transaction`	object	Contains your transaction details as recorded in the system.
`data.transaction.transaction_id`	string	Platform's unique transaction ID.
`data.transaction.external_ref`	string	Your custom transaction reference (passed during transaction creation).
`data.transaction.amount`	number	Transaction amount in base currency unit.
`data.transaction.network`	string	Network of the transaction (e.g., MTN, Airtel).
`data.transaction.msisdn`	string	Customer's phone number in MSISDN format.
`data.transaction.type`	string	Transaction type — for example, `Airtime` or `Data`.
`data.transaction.status`	string	Transaction completion status.
`data.transaction.commission_amount`	number	Commission earned from the transaction (if applicable).

`data.transaction.commission_to`	string	Indicates where the commission was credited (e.g., `Main Wallet`).
`data.transaction.refunded_at`	string / null	Timestamp of refund if applicable.
`message`	string	General message returned with the response.
`status`	string	API request status.
`status_code`	string	Response status code (`00` typically means success).

Method: GET

URL: {{host}}/transaction-by-ref/:ref

Response Example (transaction details):


```
{
 "data": {
  "telco_log": {
   "response_http_code": 200,
   "response_status_code": "0",
   "response_transaction_id": "20240517180119738072877 37",
   "status": "Successful"
  },
  "transaction": {
   "transaction_id": "3446718874516197376",
   "amount": 5000,
   "network": "mtn",
```

```
    "msisdn": "07030000000",
    "type": "Airtime",
    "external_ref": "test-mtn-may-17-3",
    "status": "Successful",
    "commission_amount": 160,
    "commission_to": "Main Wallet"
  }
 },
 "message": "Transaction Successful",
 "status": "Successful",
 "status_code": "00"
}
```


Vend Glo gift data

This endpoint allows partners to **vend Glo Gift Data bundles** directly to a subscriber's mobile number.
It supports the purchase of standard and multiple (volume-based) Glo data gift bundles.

When a transaction is initiated, the system debits the partner's account and triggers a **data gift purchase** on the Glo network. The result of the transaction is returned in real time, including the transaction ID, amount charged, bundle details, and commission earned.

Use Cases

- Automating the sale of Glo data gift bundles from an application or platform.
- Allowing end-users to gift data to others programmatically.
- Enabling corporate data gifting systems that manage bulk data distribution.

Request Body Parameters

Field	Type	Required	Description
`msisdn`	string	Yes	The Glo mobile number (MSISDN) to which the data gift will be sent. Must be a valid Glo number.
`bundle_code`	string	Yes	The unique code identifying the data bundle to be vended. You can retrieve bundle codes via the `/data-plan` endpoint.
`volume`	integer	No	Optional. The number of times the selected bundle should be applied. Default is 1. For example, a 2GB bundle with `volume: 3` will deliver 6GB total and charge three times the bundle cost.
`ref`	string	Np	Optional. A custom transaction reference provided by the client for tracking and reconciliation.

Response Parameters

Field	Type	Description
`data.transaction_id`	string	Unique identifier for the transaction.
`data.amount`	number	Total amount charged for the transaction (in Naira).
`data.network`	string	The network on which the transaction occurred (`glo`).
`data.msisdn`	string	The recipient's mobile number.
`data.type`	string	Indicates the service type — `"Data"`.
`data.data_bundle_code`	string	The code of the data bundle purchased.

`data.data_bundle_desc`	string	Human-readable description of the bundle (e.g., “3 x 40MB for N50 valid for 1 day”).
`data.external_ref`	string	The partner’s provided reference ID.
`data.status`	string	Transaction status, e.g., `"Successful"`.
`data.status_code`	string	Standardized status code returned by the system.
`data.created_at`	string	Timestamp of when the transaction was processed.
`data.balance_before`	number	Partner’s wallet balance before the transaction.
`data.commission_amount`	number	Commission amount earned on the transaction.
`data.commission_to`	string	The wallet to which the commission is credited (e.g., `"Main Wallet"`).
`data.remark`	string	Additional transaction notes (e.g., `"Volume purchased = 3"`).
`message`	string	General transaction message.
`status`	string	Overall status indicator.
`status_code`	string	API response status code.

Method: POST

URL: {{host}}/glo-gift-data

Request Body Example:

```
{
   "msisdn": "07030000000", // Glo msisdn
   "bundle_code": "A01",
   "volume": 3, // Optional. The number of times the bundle is to be applied. Default is 1. If the bundle is 2GB valid 1 month for NGN100, and you select volume of 3, NGN300 will be charged on your account, and 6GB for 1 month will be applied
   "ref": "vend_glo_1234" //Optional. Your own transaction reference.
}
```


Response Example (vend Glo gift data):


```
{
   "data": {
      "transaction_id": "3486079172502884352",
      "amount": 5000,
      "network": "glo",
      "msisdn": "07030000000",
      "type": "Data",
      "data_bundle_code": "A01",
      "data_bundle_desc": "3 X 40MB for N50 valid for 1day. ",
      "external_ref": "vend_glo_1234",
      "status": "Successful",
      "status_code": "00",
      "created_at": "2024-09-03T08:45:05.644+01:00",
      "balance_before": 531850,
      "commission_amount": 175,
      "commission_to": "Main Wallet",
      "remark": "Volume purchased = 3",
      "refunded_at": null
   },
   "message": "Transaction Successful",
   "status": "Successful",
   "status_code": "00"
}
```


Glo gift data plans

This endpoint retrieves a **list of all available Glo Gift Data bundles** that can be vended through the API.
Each data plan includes important details such as the **bundle code**, **data size**, **cost**, and **validity period**.

Developers are expected to use this endpoint to dynamically populate bundle options on their platforms or applications before initiating a data vending request via the `/glo-gift-data` endpoint.

Use Cases

- To retrieve all Glo Gift Data plans currently supported by the system.
- To display available bundles and pricing to end-users.
- To programmatically map bundle codes to their descriptions before initiating a transaction.

Method: GET

URL: {{host}}/glo-gift-plan

Response Example (glo gift data plans):


```
{
  "data": [
    {
      "code": "A01",
      "network_identifier": "49.99",
      "description": "40MB for N50 valid for 1day. ",
      "amount": 5000,
      "validity": "1 Day",
      "validity_days": 1,
      "cost": 50
    },
    {
      "code": "A02",
      "network_identifier": "99",
      "description": "100MB for N100 valid for 30day. ",
      "amount": 10000,
      "validity": "30 Day",
      "validity_days": 30,
      "cost": 100
    },
    {
      "code": "A03",
```

```
        "network_identifier": "199.03",
        "description": "200MB for N200 valid for 2 months. ",
        "amount": 20000,
        "validity": "60 Days",
        "validity_days": 60,
        "cost": 200
      }
    ]
}
```


Electricity Billers

This endpoint returns a **list of supported electricity billers** that users can pay through the API.
Each biller record includes a unique code (used for validation and vending) and a display name.

This information is essential for developers who need to populate electricity bill payment options or dynamically map biller codes for transaction processing.

Use Cases

- Display a list of available electricity distribution companies (DISCOs) to end-users.
- Retrieve biller codes to be used for bill validation and vending requests.
- Keep your platform's electricity provider list synchronized with the latest options from the API.

Response Parameters

Field	Type	Description
`data`	array	A list of electricity billers available for payment.
`data[].code`	string	The unique identifier for the biller (used in subsequent requests).
`data[].name`	string	The display name of the electricity biller.

`data[].description`	string	(Optional) Additional details about the biller.
`data[].logo_url`	string	(Optional) A URL pointing to the biller's logo for display purposes.
`message`	string	Descriptive message about the request's outcome.
`status`	boolean	Indicates the success or failure of the request.

Method: GET

URL: {{host}}/bills/electricity/billers


```
{
  "data": [
    { "name": "Abuja Electricity Distribution Company (AEDC)", "code": "ABUJA" },
    { "name": "Eko Electricity Distribution Company (EKEDC)", "code": "EKO" },
    { "name": "Ikeja Electric (IEDC)", "code": "IKEJA" },
    { "name": "Ibadan Electricity Distribution Company (IBEDC)", "code": "IBADAN" },
    { "name": "Enugu Electricity Distribution Company (EEDC)", "code": "ENUGU" }
  ],
  "message": "Successful",
  "status": true
}
```


Notes

- The `code` returned for each biller must be used in subsequent **verification** and **vending** requests (e.g., when making a payment).

- It is recommended to **cache** the billers' list locally and refresh it periodically, as billers rarely change.

- The `logo_url` field (if provided) can be used for a better visual experience in customer-facing applications.

Electricity Verify

This endpoint is used to **verify electricity bill payment details** before completing a purchase transaction.
It validates the provided **meter number**, **product type** (Prepaid or Postpaid), and optionally, the **payment amount**, and then returns details about the account or meter owner.

This helps confirm the accuracy of payment details and ensures that customers are billed correctly before vending.

Use Cases

- Verifying a customer's electricity meter number before processing payment.

- Displaying the customer's name and account details for confirmation.

- Checking applicable transaction fees and total payable amount.

Request Body Parameters

Field	Type	Required	Description
`amount`	number	No	The amount to be paid, in **kobo** (₦1 = 100 kobo). Optional — used for fee calculation.

`meter`	string	Yes	The customer's electricity **meter number** to be verified.
`product`	string	Yes	The electricity product type must be either `"Prepaid"` or `"Postpaid"`.

Response Parameters

Field	Type	Description
`data.reference_id`	string	Unique identifier for the verification request. Use this when completing the actual purchase.
`data.type`	string	Indicates the service type (e.g., `"Electricity"`).
`data.amount`	number	Amount specified for the transaction, in kobo.
`data.total_amount`	number	Total payable amount, including any applicable fees.
`data.fee`	number	The transaction or service fee charged.
`data.biller`	string	The electricity distribution company name.

`data.biller_product`	string	The electricity product selected (Prepaid/Postpaid).
`data.beneficiary_number`	string	The verified meter number.
`data.beneficiary_name`	string	Name of the customer linked to the meter.
`data.beneficiary_details`	string	Additional customer information (e.g., address).
`message`	string	Status message describing the outcome of the verification.
`status`	boolean	Indicates whether the verification was successful.

Method: POST

URL:`{{host}}/bills/electricity/verify`

Example Request


```
{
  "amount": 100000,
  "meter": "1234567890",
  "product": "Prepaid"
}
```

Example Response (Successful Verification)


```
{
    "data": {
        "reference_id": "3602804885318012928",
        "type": "Electricity",
        "amount": 100000,
        "total_amount": 105000,
        "fee": 5000,
        "biller": "Electricity",
        "biller_product": "Prepaid",
        "beneficiary_number": "1234567890",
        "beneficiary_name": "Jon Doe",
        "beneficiary_details": "Jon Doe Address"
    },
    "message": "Successful",
    "status": true
}
```


Example Response (Validation Error)


```
{
    "error": "Amount must be between NGN 500 and NGN 100,000",
    "status": false
}
```

Notes

- Always perform a verification before attempting to vend electricity to ensure customer and meter details are valid.
- The **reference_id** returned from this endpoint can be used when initiating the **actual electricity purchase**.
- If the amount field is omitted, the system may still return customer details, but fees might not be calculated.
- Ensure that the product field strictly matches either "Prepaid" or "Postpaid" (case-sensitive).
- The API enforces **minimum and maximum payment limits** based on the biller's configuration.
- For failed verifications, check the error or errors object for more details about what went wrong.

Electricity Vend

URL: `{{host}}/bills/electricity/vend`

Method: POST

This endpoint allows you to **vend (purchase)** electricity tokens for **Prepaid** or **Postpaid** meters.
It completes the electricity payment process after a successful **verification** via the `/bills/electricity/verify` endpoint.

The response contains full transaction details, including the **token** (for Prepaid meters), **transaction status**, and **timestamps**.

Use Cases

- To complete an electricity bill payment after verification.
- To generate prepaid electricity tokens for customers.
- To credit postpaid customer accounts with validated payments

Request Body Parameters

Field	Type	Required	Description
`amount`	number	Yes	Amount to be paid, in **kobo** (₦1 = 100 kobo).
`biller`	string	Yes	The electricity provider's **code** (retrieved from the `/bills/electricity/billers` endpoint).
`meter`	string	Yes	The customer's **meter number**.
`product`	string	Yes	The electricity product type — must be either `"Prepaid"` or `"Postpaid"`.
`name`	string	Yes	The beneficiary's name, as returned by the `/verify` endpoint.
`address`	string	Yes	The beneficiary's address, also returned from verification.

`ref`	string	No	Optional custom transaction reference for reconciliation and tracking.

Example Request

```
{
    "amount": 100000,
    "biller": "EKO",
    "meter": "0101150481949",
    "product": "Prepaid",
    "name": "ORITSE EBINEMI",
    "address": "Alhaji Marufu Ali Owe Street Plt8b Agungi Lekki",
    "ref": "123456789000"
}
```

Response Parameters

Field	Type	Description
`data.transaction_id`	string	Unique transaction identifier for the electricity vend.
`data.type`	string	Transaction type — `"Electricity"`.
`data.amount`	number	Transaction amount (in kobo).

`data.fee`	number	Transaction fee applied, if any.
`data.biller`	string	Electricity provider's name.
`data.biller_code`	string	The unique code representing the electricity provider.
`data.biller_product`	string	Product type (Prepaid or Postpaid).
`data.beneficiary_number`	string	Beneficiary's meter number.
`data.beneficiary_name`	string	Customer's name.
`data.beneficiary_details`	string	Customer's address or related details.
`data.status`	string	Transaction status — e.g., `"Successful"` or `"Failed"`.
`data.status_code`	string	Standardized code for transaction outcome.
`data.voucher`	string	Electricity **token** (for Prepaid customers).
`data.instruction`	string	Vendor instruction or notes (if any).

`data.balance_before`	number	Partner's wallet balance before transaction.
`data.created_at`	string	Timestamp when the transaction was processed.
`message`	string	Descriptive message about the result.
`status`	string	Overall API status — `"Successful"` or `"Failed"`.

Example Response (Successful Vend)


```
{
    "data": {
        "transaction_id": "3602817395572084736",
        "amount": 100000,
        "fee": 0,
        "biller": "EKO",
        "type": "Electricity",
        "status": "Successful",
        "status_code": "00",
        "beneficiary_name": "Jon Doe",
        "beneficiary_details": "Jon Doe Address",
        "token": "0028-7952-0226-6061-7666",
        "created_at": "2025-07-22T12:01:06.914+01:00"
```

```
    },
    "message": "Transaction Successful",
    "status": "Successful",
    "status_code": "00"
}
```

Notes

- Always call `/bills/electricity/verify` **before vending**, to confirm meter validity and retrieve beneficiary information.
- Ensure your wallet balance is sufficient for the **total amount + fees**.
- For **Prepaid** customers, the `token` value in the response represents the electricity recharge code.
- For **Postpaid** customers, the transaction will post a bill payment but **no token** will be returned.
- Store the returned `transaction_id` for reconciliation and audit purposes.
- Use `/transaction/:transactionID` or `/transaction-by-ref/:ref` to check the final status of any vend transaction.
- If the transaction fails, verify the input parameters and ensure the meter number and biller code are correct.

Cable TV Billers

Method: `GET`
URL: `{{host}}/bills/cabletv/billers`

Description

This endpoint retrieves a list of **supported Cable TV service providers** (billers) available on the platform.
Each biller entry includes its unique **code** and **display name**, which must be used when performing verification or vending operations.

Developers can use this endpoint to dynamically populate provider lists in customer-facing applications.

Use Cases

- Display available Cable TV providers (e.g., DSTV, GOTV, Startimes).
- Retrieve the correct `biller code` to be used for verification and payment transactions.
- Keep the list of available TV providers synchronized with the latest system updates.

Response Parameters

Field	Type	Description
`data`	array	A list of all available Cable TV billers.
`data[].code`	string	Unique identifier for the Cable TV provider.
`data[].name`	string	Display name of the Cable TV provider.
`data[].description`	string	(Optional) Details about the provider.

`data[].logo_url`	string	(Optional) URL to the provider's logo.
`message`	string	Descriptive message indicating request status.
`status`	boolean	Indicates whether the request was successful.

Example Response


```
{
    "data": [
        { "name": "DSTV", "code": "DSTV" },
        { "name": "GOTV", "code": "GOTV" },
        { "name": "Startimes", "code": "STARTIMES" }
    ],
    "message": "Successful",
    "status": true
}
```

Notes

- Use the `code` field when verifying or vending a subscription.
- Billers occasionally update product bundles — ensure you refresh this list periodically.
- This endpoint does not require authentication beyond your API key or token.

Cable TV Biller Products

Method: `GET`
URL: `{{host}}/bills/cable/biller-products?biller=GOTV`

Description

This endpoint retrieves the list of **available subscription products or packages** (bouquets) for a specific Cable TV provider.
Each product includes a unique **code**, **name**, and **amount**, which are required when vending a subscription.

This enables developers to present customers with valid package options such as **DSTV Premium**, **GOTV Max**, or **Startimes Nova**.

Use Cases

- Fetch available subscription packages for a selected Cable TV provider.
- Populate dropdowns or menus with up-to-date pricing and product names.
- Ensure the correct product code is used when initiating a vending transaction.

Query Parameters

Parameter	Type	Required	Description
`biller`	string	Yes	The code of the Cable TV provider (e.g., `"DSTV"`, `"GOTV"`, `"STARTIMES"`).

Response Parameters

Field	Type	Description

`data`	array	List of subscription products for the specified provider.
`data[].code`	string	Unique code of the subscription product — required for vending.
`data[].name`	string	Display name of the subscription product.
`data[].amount`	number	Subscription price (in **kobo**).
`message`	string	Descriptive status message of the request.
`status`	boolean	Indicates if the request was successful.

Example Response

```
{
    "data": [
        { "code": "1234", "name": "Product One", "amount": 307000
},
        { "code": "233344", "name": "Product 2", "amount":
2400000 }
    ],
    "message": "Successful",
    "status": true
}
```

Notes

- Always specify the `biller` query parameter when calling this endpoint.

- Product codes retrieved here are required in the **Cable TV Verification** and **Vending** endpoints.
- The `amount` value is in **kobo**; divide by 100 to display in naira.
- Refresh this list periodically to ensure that your system reflects the most recent product lineup and prices.

Cable TV Verification

Method: `POST`
URL: `{{host}}/bills/cabletv/verify`

Description

This endpoint allows you to **verify a Cable TV subscriber's details** before completing a payment.
It validates the **smartcard number** or **account number** and confirms customer information such as the subscriber's name and current subscription plan.

Verification ensures that the payment is made to the correct account and helps prevent errors before vending.

Use Cases

- Confirm a subscriber's smartcard number and account validity.
- Retrieve subscriber details such as name and product type before payment.
- Calculate total payable amount (including service fees) before vending.

Request Body Parameters

Field	Type	Required	Description
`smartcard_number`	string	Yes	The customer's smartcard or account number.
`biller`	string	Yes	The code of the Cable TV provider (e.g., `"DSTV"`, `"GOTV"`, `"STARTIMES"`).
`product_code`	string	No	Optional. The specific product or bouquet code to verify, if applicable.

Example Request

```
{
    "smartcard_number": "7029374012",
    "biller": "DSTV"
}
```

Response Parameters

Field	Type	Description
`data.reference_id`	string	Unique reference ID for the verification request.
`data.type`	string	The service type — `"CableTV"`.
`data.biller`	string	The Cable TV provider's name.
`data.biller_product`	string	The product or bouquet type.

`data.beneficiary_number`	string	The verified smartcard or account number.
`data.beneficiary_name`	string	The subscriber's registered name.
`data.beneficiary_details`	string	Additional account or plan information.
`message`	string	Description of the verification result.
`status`	boolean	Indicates if verification was successful.

Example Response (Successful Verification)

```
{
    "data": {
        "reference_id": "4701827361948203",
        "type": "CableTV",
        "biller": "DSTV",
        "biller_product": "Premium",
        "beneficiary_number": "7029374012",
        "beneficiary_name": "John Doe",
        "beneficiary_details": "Active Subscription: Premium
Plan"
    },
    "message": "Verification Successful",
```

```
    "status": true
}
```

Example Response (Invalid Smartcard)

```
{
    "error": "Invalid Smartcard Number",
    "status": false
}
```

Notes

- Always verify the customer's smartcard or account before vending.
- The returned `reference_id` must be used when completing the **vending transaction**.
- For DSTV and GOTV, ensure the smartcard number corresponds to the correct provider code.
- Handle verification failures gracefully — prompt the user to recheck their account number.
- The API may return additional information such as bouquet options or account status, depending on the provider.

Cable Vend Service

Method: `POST`
URL: `{{host}}/bills/cable/vend`

Description

The **Cable Vend Service** endpoint allows you to **purchase or renew Cable TV subscriptions** by submitting a POST request with the necessary billing and customer details.

It completes the vending process for Cable TV providers such as **DSTV**, **GOTV**, or others, after verification. The API returns a detailed transaction record, including the vending status, bouquet information, and transaction timestamps.

Use Cases

- To activate or renew a Cable TV package for a verified customer.
- To automate subscription vending through partner integrations.
- To process one-time or recurring Cable TV payments.

Request Body Parameters

Field	Type	Required	Description
`amount`	integer	Yes	The total amount of the subscription, in **kobo** (₦1 = 100 kobo).
`biller`	string	Yes	The name or code of the Cable TV provider (e.g., `"DSTV"`, `"GOTV"`).
`number`	string	Yes	The customer's **Smart Card Number** or account identifier.

`product`	string	Yes	The product code obtained from `/bills/cable/biller-products`.
`name`	string	Yes	The beneficiary's name, returned from the verification endpoint.
`address`	string	Yes	The beneficiary's address, also from the verification response.
`ref`	string	No	Optional custom transaction reference, used for tracking and reconciliation.

Example Request

```
{
    "biller": "GOTV",
    "number": "8072156259",
    "product": "ng_gotv_gotvnj1",
    "name": "FATIMA HUSSEIN",
    "address": "8072156259",
    "ref": "123456789000"
}
```

Response Parameters

Field	Type	Description
`data.transaction_id`	string	Unique identifier assigned to the vending transaction.

`data.type`	string	Type of transaction, e.g., `"Cable"`.
`data.amount`	number	Amount charged (in kobo).
`data.fee`	number	Transaction fee charged, if applicable.
`data.biller`	string	The name of the Cable TV provider.
`data.biller_product`	string	The product or bouquet name.
`data.beneficiary_number`	string	The smart card number or account number of the customer.
`data.beneficiary_name`	string	The name of the subscriber.
`data.beneficiary_details`	string	Additional information such as address or plan details.
`data.status`	string	Status of the transaction — `"Successful"`, `"Pending"`, or `"Failed"`.
`data.status_code`	string	Standardized result code for the transaction.
`data.data_bundle_desc`	string	Description of the bouquet or service purchased.
`data.created_at`	string	Date and time when the transaction was processed.
`message`	string	General response message about the transaction.
`status`	string	Overall vending outcome — `"Successful"` or `"Failed"`.

Example Response (Successful Vend)


```
{
    "data": {
        "transaction_id": "3543106302037987328",
        "amount": 60000,
        "network": "GOTV",
        "type": "Cable",
        "data_bundle_code": "ng_gotv_gotvnj1",
        "data_bundle_desc": "GOtv Jinja_ valid for 1 month",
        "status": "Successful",
        "status_code": "00",
        "created_at": "2025-02-07T17:30:32.844+01:00",
        "balance_before": 1034050
    },
    "message": "Transaction Successful",
    "status": "Successful",
    "status_code": "00"
}
```

Example Response (Failed Vend)


```
{
    "message": "Invalid Smart Card Number or Unavailable
Product",
```

```
    "status": "Failed"
}
```

Notes

- Always perform **verification** using `/bills/cabletv/verify` before vending to confirm customer and product validity.
- Use the correct `product` code retrieved from `/bills/cable/biller-products` to avoid vending errors.
- The `ref` field helps prevent duplicate transactions — use a unique reference for each vend.
- If you receive a `"Pending"` status, query the transaction later using `/transaction/:transactionID` or `/transaction-by-ref/:ref`.
- The `data_bundle_desc` field contains human-readable bouquet details, ideal for receipts or UI display.
- Ensure your system logs both `transaction_id` and `ref` for reconciliation and support.

Balance Check

Method: `GET`
URL: `https://api.airtimereport.com/api/me/balance`

Description:
This endpoint allows developers to retrieve the current wallet balance associated with their API account. The wallet balance is required to validate whether sufficient funds are available before performing transactions such as airtime, data, or utility vending.

Request

No request body is required.
The request must include valid authentication headers as specified in the **Authentication** section.

Response

Response Example (Successful):

```
{
  "data": {
    "balance": 100000,
    "commission": 500,
    "bonus": "0
  },
  "message": "Successful",
}
```

Response Example (Unauthorized):

```
{
  "message": "Unauthorized",
  "status": false
}
```

Field Descriptions

Field	Type	Description

`balance`	number	The available balance in the main wallet (used for transactions).
`commission`	number	The available balance in the commission wallet (earned commissions).
`bonus`	number	The available bonus balance
`message`	string	Descriptive message about the response.

Usage Notes

- Always perform a **balance check** before initiating high-value or multiple transactions.
- For best performance, cache the balance value for short periods (e.g., 60 seconds) instead of querying on every request.
- Use the balance to ensure that transaction requests do not exceed available funds.

4. Example Integration Flow

A typical integration with the Hisa Go Partner API follows these steps:
1. Authenticate and retrieve an access token.
2. Use the token to make authorized requests to endpoints such as airtime verification or transaction submission.
3. Handle responses and error codes as documented.

5. Error Codes and Troubleshooting

Common HTTP error responses include:
• 400 Bad Request – The request is invalid or missing parameters.
• 401 Unauthorized – Authentication failed or missing credentials.
• 403 Forbidden – You are not authorized to access this resource.

• 404 Not Found – The requested resource does not exist.
• 500 Internal Server Error – The server encountered an unexpected condition.

6. Frequently Asked Questions (FAQ)

Note: All amounts to and from the system are in kobo

- Q: How do I obtain my API key?

A: Contact the platform administrator or support to request API credentials.

- Q: What format should requests and responses use?

A: All requests and responses use JSON format over HTTPS.

- Q: How do I handle timeout errors?

A: When a timeout occurs, first check the transaction status using your unique reference via the `GET /transaction-by-ref/:ref` endpoint to confirm whether the transaction succeeded or failed.

If the transaction is not found or remains pending after a reasonable delay, implement retry logic with **exponential backoff** to avoid overloading the system and prevent duplicate charges.

- Q: Are these credentials for a sandbox or live environment?

A: These credentials are on sandbox, but perform live transactions to make integration easier. You have NGN 5000 for testing, and the recipients will get the actual value; therefore, perform each test with small amounts. Remember, all amounts to and from the system are in Kobo. Once you are done with testing, a different API key and URL will be used for the Live environment. But your email and password remain the same (you can choose to request a change).

7. Support and Contact

For further technical assistance, please reach out to the support team or your assigned technical contact.